TABLE OF CONTENTS

1. INTRODUCTION

Introduction to Artificial Intelligence and Assignment Objective

Nowadays, we are frequently listening,speaking and talking about Artificial Intelligence or AI whatever it is, but do you what it is. Artificial Intelligence is like a human brain but feed with all the information floating out in the world so that it can act as a human being, but have you ever wondered how it became so intelligent and how it is trained. Let's deep dive into it the research and explore all about it.

The main objective of this report to grasp the main concept of AI by making a model to detect the credit card defaults by analyzing all the previous data about the transaction we have, balance summary and bills. Now after analyzing we will able to detect some trends and see if the data we are giving to AI is good or bad, by looking over this we see this is a classification or regression and check for models which suits best for our model and then train the model and lastly we see how our model is performing.

2. METHODOLOGY AND IMPLEMENTATION

2.1 Exploratory Data Analysis

Before starting, lets import some libraries or modules which are important and going to help us throughout model building such as pandas, numpy, matplotlib.pyplot, Standard Scaler, Power Transformer, train_test_split, Simple Imputer, Column Transformer, Pipeline, accuracy_score, classification_report, confusion_matrix, auc, roc_curve.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler, PowerTransformer
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import auc, roc_curve
```

Let's parse our dataset using pd.read_csv('Dataset Address').

```
df_original = pd.read_csv('/content/drive/MyDrive/final assignment data/Credit_Card.csv',sep=';',na_values = '?',low_memory=False)
df_copy = df_original.copy()
df_copy.head()
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6	default.payment.next.month	risk_leak	BILL_AMT_SUM	LIMIT_BAL_LOG	CITY	RISK_RATING
0	1	20000.0	2.0	2.0	NaN	24.0	2	2	-1	-1	...	0.0	0.0	0.0	0.0	1	1.075.675.276.433.290	7704.0	990.353.755.128.617	City_38	2
1	2	120000.0	2.0	2.0	2.0	26.0	-1	2	0	0	...	1000.0	1000.0	0.0	2000.0	1	10.940.840.854.872.400	17077.0	11.695.255.355.062.700	City_6	1
2	3	90000.0	2.0	2.0	2.0	34.0	0	0	0	0	...	1000.0	1000.0	1000.0	5000.0	0	0.0703022492301175	101653.0	11.407.576.060.361.700	City_20	1
3	4	50000.0	2.0	2.0	1.0	37.0	0	0	0	0	...	1200.0	1100.0	1069.0	1000.0	0	0.0584424354671778 46	231334.0	10.819.798.284.210.200	City_25	1
4	5	50000.0	1.0	2.0	1.0	57.0	-1	0	-1	0	...	10000.0	9000.0	689.0	679.0	0	0.12943102887089503	109339.0	10.819.798.284.210.200	City_44	1

5 rows × 30 columns

Now we have our dataset parsed lets do a exploratory data analysis of our dataset.

```
print(df_copy.shape)

print('\nColumn names: ')
print(df_copy.columns.tolist())

print('\nData types: ')
display(df_copy.dtypes.to_frame(name='Data Type'))

print('\nMissing values: ')
print(df_copy.isna().sum().to_frame(name='Missing Values'))

print('\nNumber of duplicate rows: ',df_copy.duplicated().sum())

display(df_copy.describe())
```

During the Analysis we found that this dataset has 30 columns and 34788 rows. Most of the columns have a floating point data and integer based data, only city column is object type data but there is one problem we see risk_leak and LIMIT_BAL_LOG is parsed as object as per our guidelines it is a floating point values so it is a 1st issue. And we now move on and check for missing values and duplicated rows in our dataset and we find there are 4788 duplicated rows which is 2nd issue and also we see there are missing values in LIMIT_BAL, SEX, EDUCATION, MARRIAGE, AGE, PAY_AMT1, PAY_AMT2, LIMIT_BAL_LOG near about 1700 in each case which is also our 3rd issue. Lastly, after visualization we find in some parts there is a right skewness and plenty of outliers which is our 4th and 5th issue in this analysis.

2.2 Data Preparation

Now in data preparation we will resolve our problems we found during data analysis.

1st Issue Parsing float value to object type:

```
#ISSUE NUMBER 1
issue_1 = df_copy[['risk_leak','LIMIT_BAL_LOG']]
issue_1.head()
```

	risk_leak	LIMIT_BAL_LOG
0	1.075.675.276.433.290	990.353.755.128.617
1	10.940.840.854.872.400	11.695.255.355.062.700
2	0.0703022492301175	11.407.576.060.361.700
3	0.058442435467177846	10.819.798.284.210.200
4	0.12943102887089503	10.819.798.284.210.200

Here we see it is not the correct way to write floating point value as it has to many points in each value and that's why pandas parse these two columns as a object and we cannot turn it back it into floating point value it will throw an error but we use one more method i.e. df_copy['risk_leak'] = pd.to_numeric(df_copy['risk_leak'],

errors='coerce') but this method only works with risk_leak and on the other hand it turns all the value of LIMIT_BAL_LOG to null value so it is no more useful to us that's why we will ignore this and also we don't have any other method to turn this.

2nd Issue duplicated rows:

```
#ISSUE NUMBER 2
duplicate_rows = df_copy[df_copy.duplicated()]
print('\nNumber of duplicate rows: ',df_copy.duplicated().sum())
display(duplicate_rows.head())
```

```
Number of duplicate rows:  4788
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6	default.payment.next.month	risk_leak	BILL_AMT_SUM	LIMIT_BAL_LOG	CITY	RISK_RATING
30000	23199	390000.0	2.0	2.0	2.0	39.0	0	0	0	0	...	3000.0	3000.0	5000.0	3000.0	0	0.011421	339967.0	12.873.904.582.205.100	City_15	1
30001	358	380000.0	1.0	NaN	2.0	34.0	0	0	0	0	...	5000.0	6000.0	5000.0	4400.0	0	0.050856	1082561.0	12.847.929.163.278.000	City_2	1
30002	20580	50000.0	1.0	3.0	2.0	27.0	-1	-1	-1	-1	...	21423.0	6308.0	12651.0	6243.0	0	-0.140022	53632.0	10.819.798.284.210.200	City_13	1
30003	24303	100000.0	1.0	1.0	NaN	31.0	2	2	2	2	...	4005.0	600.0	2400.0	2900.0	1	0.954341	556488.0	1.151.293.546.492.020	City_12	2
30004	24432	70000.0	1.0	2.0	2.0	39.0	0	0	0	0	...	1500.0	1200.0	1500.0	1500.0	1	0.873273	310992.0	11.156.264.806.643.700	City_32	1

5 rows × 30 columns

There are 4788 duplicated rows and most reliable solution is to delete all the duplicate rows as there is same value already present in the dataset and the duplicated values only increase biasness in the dataset.

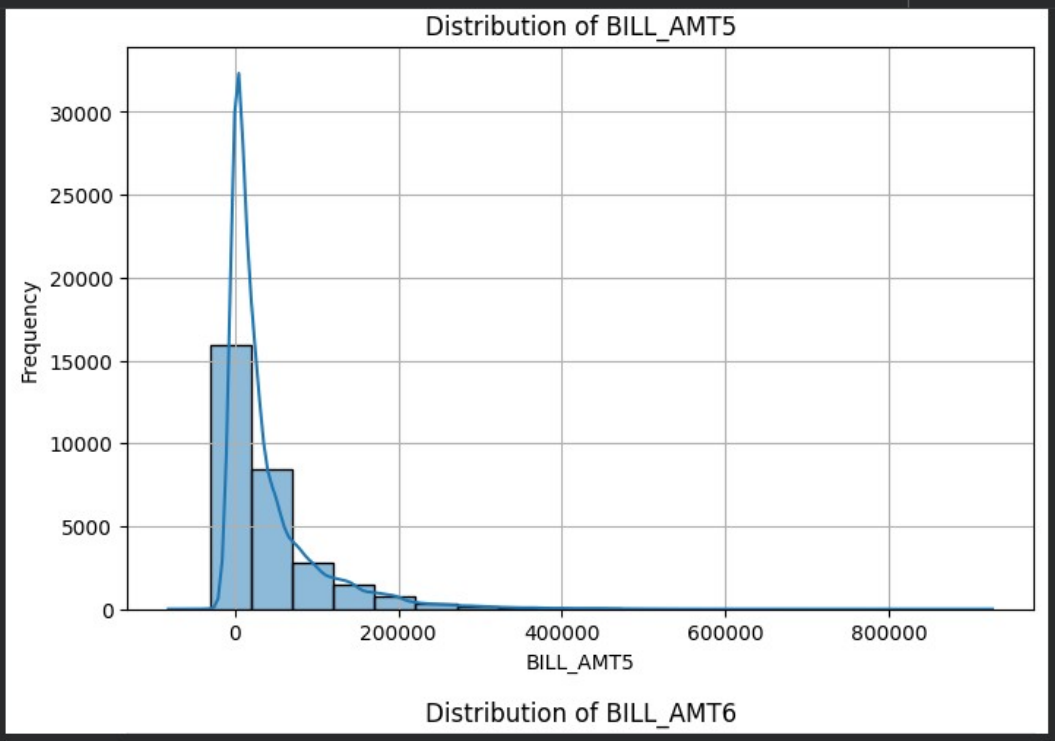
3rd , 4th and 5th Issue which is about Missing Value, Skewness and Outliers are solved in a same manner:

```
#ISSUE NUMBER 3
print('\nMissing values: ')
print(df_copy.isna().sum().to_frame(name='Missing Values'))
```

```
Missing values:
                            Missing Values
ID                                       0
LIMIT_BAL                             1500
SEX                                   1500
EDUCATION                             1500
MARRIAGE                              1500
AGE                                   1500
PAY_0                                    0
PAY_2                                    0
PAY_3                                    0
PAY_4                                    0
PAY_5                                    0
PAY_6                                    0
BILL_AMT1                                0
BILL_AMT2                                0
BILL_AMT3                                0
BILL_AMT4                                0
BILL_AMT5                                0
BILL_AMT6                                0
PAY_AMT1                              1500
PAY_AMT2                              1500
PAY_AMT3                                 0
PAY_AMT4                                 0
PAY_AMT5                                 0
PAY_AMT6                                 0
default.payment.next.month               0
risk_leak                             3371
BILL_AMT_SUM                             0
LIMIT_BAL_LOG                         1500
CITY                                     0
RISK_RATING                              0
```

Missing values in all the columns

```python
import seaborn as sns

Numerical_features = ['LIMIT_BAL','PAY_0','PAY_2','PAY_3','PAY_4','PAY_5','PAY_6',
                      'BILL_AMT1','BILL_AMT2','BILL_AMT3','BILL_AMT4','BILL_AMT5','BILL_AMT6',
                      'PAY_AMT1','PAY_AMT2','PAY_AMT3','PAY_AMT4','PAY_AMT5','PAY_AMT6','risk_leak',
                      'BILL_AMT_SUM','RISK_RATING'
                      ]

for column in Numerical_features:
    plt.figure()
    sns.histplot(df_copy[column], bins=20, kde=True, edgecolor="black")
    plt.title(f"Distribution of {column}")
    plt.xlabel(column)
    plt.ylabel("Frequency")
    plt.show()
```

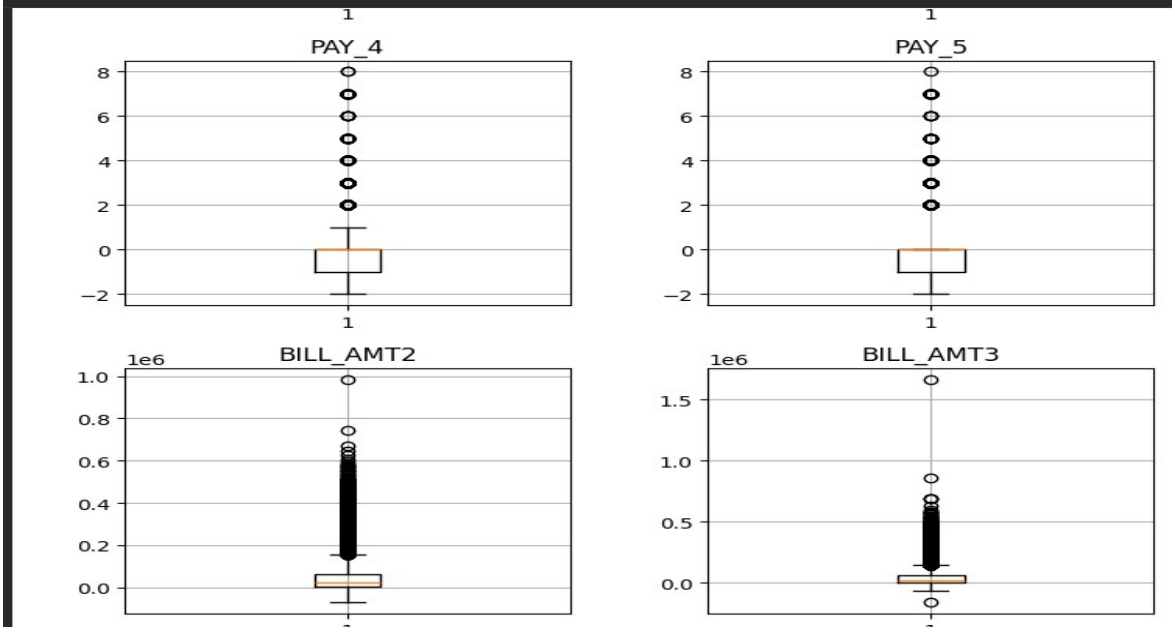


Right Skewness in some features

```
Q1 = df_copy[Numerical_features].quantile(0.25)
Q3 = df_copy[Numerical_features].quantile(0.75)
IQR = Q3 - Q1
outliers_count_specified = ((df_copy[Numerical_features] < (Q1 - 1.5 * IQR)) | (df_copy[Numerical_features] > (Q3 + 1.5 * IQR))).sum()
outliers_count_specified
```

	0
LIMIT_BAL	390
PAY_0	3130
PAY_2	4410
PAY_3	4209
PAY_4	3508
PAY_5	2968
PAY_6	3079
BILL_AMT1	2400
BILL_AMT2	2395
BILL_AMT3	2469
BILL_AMT4	2622
BILL_AMT5	2725
BILL_AMT6	2693
PAY_AMT1	2607
PAY_AMT2	2552
PAY_AMT3	2598
PAY_AMT4	2994
PAY_AMT5	2945
PAY_AMT6	2958
risk_leak	3326
BILL_AMT_SUM	2645
RISK_RATING	6818

```
plt.figure(figsize=(15, 18))
for i, col in enumerate(Numerical_features):
    plt.subplot(6, 4, i + 1)
    plt.boxplot(df_copy[col])
    plt.title(col)
plt.tight_layout()
plt.show()
```



These 2 figures shows the outliers by Visualization & by Data points

Firstly we check outliers and as we see it by data points and visualization there are 2500 to 7000 outliers present column wise in the figure but we our not going to squeeze or minimize its effect as each value as a outlier represent a amount like bill amount. We get to this conclusion because when we read the description of the data set we also see every outlier is necessary in this model cannot be deleted or minimized.

Now lets talk about skewness and missing value, yes there are some skewness and missing values in some columns but as per our inspection we our going to drop these columns ID, SEX, EDUCATION, MARRIAGE, AGE, CITY, LIMIT_BAL_LOG as these feature are not that much important features need to be train to our model and then we build a pipeline after splitting our dataset in 80:20 ratio to handle this problem by Simple Imputer and yeo-johnson, yeo-johnson is used because there are some negative values also so to treat them we will not use box-cox. While building pipeline for missing value and skewness we see there are only numerical patterns left here in our dataset on which we are going to train our model, so we use Standard Scaler to scale all the data points.

2.3 Model Training

```
X = df_copy.drop(['default.payment.next.month','ID', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE','CITY','risk_leak','LIMIT_BAL_LOG'], axis=1)
y = df_copy['default.payment.next.month']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
X_train_fe = X_train.copy()
X_test_fe = X_test.copy()

Numerical_features = X_train_fe.select_dtypes(include=["number"]).columns.tolist()
print(Numerical_features)

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("yeo-johnson", PowerTransformer(method="yeo-johnson",standardize=False)),
    ("scaler", StandardScaler())
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, Numerical_features)
])
```

In this training part what we do is we remove the target variable, and ID, SEX, EDUCATION, MARRIAGE, AGE, CITY, LIMIT_BAL_LOG as these are not necessary independent variables on which we want to train our model also we drop risk_leak because it totally aligns with dependent variable and may overfit the

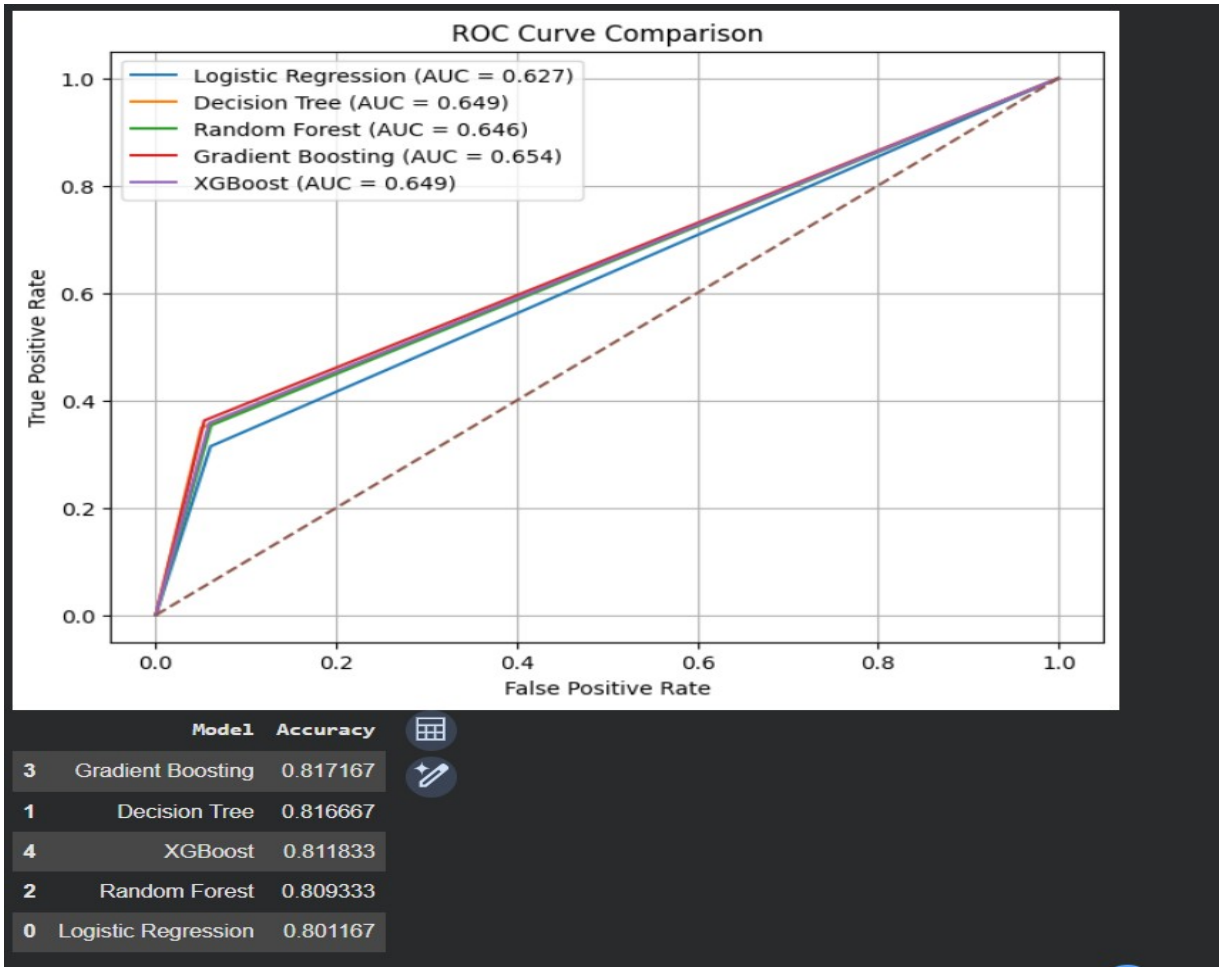
training dataset and accuracy with this is 0.99 and then we split our data in 80:20 ration, 80% for training part and 20% for testing part also we use stratify to get all the classes in both the sets and randomizer for random row values in the sets.

After the training part, we build pipeline to solve our issue 3 and issue 4 which we already discuss in the upper part. Now let's jump to model selection and train our model.

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier

log_reg_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])

tree_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DecisionTreeClassifier(random_state=42, max_depth=4))
])

forest_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(random_state=42))
])

gb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", GradientBoostingClassifier(random_state=42))
])

xgb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", XGBClassifier(random_state=42))
])

log_reg_model.fit(X_train_fe, y_train)
tree_model.fit(X_train_fe, y_train)
forest_model.fit(X_train_fe, y_train)
gb_model.fit(X_train_fe, y_train)
xgb_model.fit(X_train_fe, y_train)
```

Here we use 5 models to train i.e. Logistic Regression, Decision Tree Classifier, Random Forest Classifier, Gradient Boosting Classifier, XGB Classifier and then we proceed with our pipeline next steps and train our model.

3. MODEL EVALUATION AND VISUALIZATION



	Model	Accuracy
3	Gradient Boosting	0.817167
1	Decision Tree	0.816667
4	XGBoost	0.811833
2	Random Forest	0.809333
0	Logistic Regression	0.801167

```
Best model: Gradient Boosting

Classification report:
              precision    recall  f1-score   support

           0       0.84      0.95      0.89      4673
           1       0.66      0.36      0.47      1327

    accuracy                           0.82      6000
   macro avg       0.75      0.65      0.68      6000
weighted avg       0.80      0.82      0.80      6000

Confusion matrix:
[[4422  251]
 [ 846  481]]
```

Here we see accuracy and all other matrix to evaluate and visualize which model is good for use and therefore we come up with a conclusion to use Gradient Boosting model which has a accuracy of 0.817167 which is best in the all the five models.

4. CONCLUSION

To conclude with the model we start from data preparation where we prepare our data against different problems and train our model with five different model which are Logistic Regression, Decision Tree Classifier, Random Forest Classifier, Gradient Boosting Classifier, XGB Classifier, from all of these Gradient Boosting Classifier gives maximum accuracy of 0.817167 and on the other hand Logistic Regression gives the lowest accuracy of 0.801167